Centro de Monitoreo de Accesos

Actividad Evaluativa #1 — Programación Orientada a Objetos

2.º curso Ciberseguridad | ENTI – Universitat de Barcelona

23 de octubre de 2026

1 Introducción y Enfoque Arquitectónico

El objetivo de esta práctica consiste en implementar un sistema robusto que permita registrar, auditar y clasificar intentos de acceso a dos servidores de red independientes, identificando cuál de ellos requiere atención prioritaria por parte del equipo de ciberseguridad.

Siguiendo estrictamente las restricciones curriculares (sin uso de arrays, colecciones ni herencia), la solución se organiza en un único archivo fuente denominado `Main.java`, estructurado en torno a dos clases principales:

- **Clase `Servidor`:** Núcleo de la lógica de negocio. Es la encargada de representar el estado de cada máquina mediante los cinco atributos de instancia requeridos (`nombre`, `origen`, `critico`, `intentosTotales`, `intentosFallidos`). Concentra de forma autónoma la validación de datos, el cálculo porcentual y la evaluación de riesgo.
- **Clase `Main`:** Contiene el método de ejecución principal, gestiona la captura interactiva de datos mediante la clase `Scanner`, instancia dos objetos independientes y realiza la comparativa final de criticidad entre ambos sistemas.

2 Diseño de Métodos y Lógica de Seguridad

2.1 Validación y Sanitización de Datos (`datosValidos`)

Antes de ejecutar cualquier cómputo aritmético o clasificación de riesgo, el sistema verifica la coherencia del estado del servidor:

- **Tolerancia y normalización de texto:** Se utiliza `trim()` para eliminar espacios sobrantes al inicio y final de las cadenas, combinándolo con `equalsIgnoreCase()` para aceptar entradas robustas en mayúsculas, minúsculas o con espacios intermedios (ej. entradas como `"EXTERNO ␣ "SI""`).
- **Consistencia numérica:** Se comprueba que `intentosTotales > 0` y que los fallos se encuentren dentro del intervalo cerrado $[0, \text{intentosTotales}]$. En caso de detectarse cualquier anomalía, el servidor es clasificado de inmediato como `DATOS INVALIDOS`.

2.2 Cálculo Seguro del Porcentaje (`porcentajeFallos`)

Para prevenir el truncamiento numérico derivado de la división entera nativa de Java, se realiza una coerción explícita de los tipos enteros a número real (`double`):

$$\text{Porcentaje} = \left(\frac{\text{intentosFallidos}}{\text{intentosTotales}} \right) \times 100.0$$

Si el objeto evaluado contiene datos inválidos, el método de resumen omite deliberadamente la operación aritmética y retorna `N/D`, evitando divisiones por cero o resultados inconsistentes.

2.3 Jerarquía de Clasificación y Prioridad

El método `evaluarEstado()` clasifica la amenaza respetando un orden de precedencia estricto:

1. **BLOQUEADO:** Se asigna si los fallos alcanzan o superan el umbral de 5, o bien si el servidor es crítico, de acceso externo y registra al menos 3 fallos.
2. **ALERTA:** Se asigna si registra 3 o 4 fallos, o si es de acceso externo con 2 o más fallos (siempre que no cumpla las condiciones de bloqueo).
3. **NORMAL:** Cualquier servidor con datos válidos que no incurra en las categorías anteriores.

En la comparativa final entre ambos servidores se aplica una escala de prioridad jerárquica: BLOQUEADO $>$ ALERTA $>$ NORMAL (arrojando `COMPARACION NO DISPONIBLE` si alguno de los nodos presenta datos corruptos).

3 Validación y Casos de Prueba

Se verificó el comportamiento del programa frente a la batería completa de casos de prueba obligatorios, confirmando una correspondencia exacta con la salida esperada:

Servidor	Origen	Crítico	Total	Fallos	Porcentaje	Estado Obtenido
VPN	externo	sí	10	3	30.0%	BLOQUEADO
WEB	interno	no	8	4	50.0%	ALERTA
CORREO	externo	no	8	2	25.0%	ALERTA
VPN	interno	sí	12	2	16.7%	NORMAL
WEB	externo	no	5	5	100.0%	BLOQUEADO
WEB	interno	no	4	5	N/D	DATOS INVALIDOS
WEB	interno	no	0	0	N/D	DATOS INVALIDOS
CORREO	local	no	10	2	N/D	DATOS INVALIDOS
WEB	quizás	externo	10	2	N/D	DATOS INVALIDOS

4 Conclusiones

La solución desarrollada cumple formalmente con todas las restricciones del enunciado. Al encapsular toda la lógica de validación y cálculo de riesgo dentro de la propia clase `Servidor`, se logra un diseño modular, altamente limpio, tolerante a entradas defectuosas y de gran robustez técnica para entornos de ciberseguridad.